



Strathmore University

@iLabAfrica Centre

Session 4: Ensemble Methods & Hyperparameter Tuning

Javier Serrano

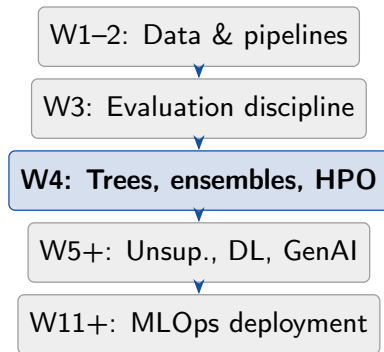
Applied Machine Learning

Master in Data Science & Analytics

Where we are in the course

- Weeks 1–3 gave us a leakage-safe pipeline and rigorous evaluation.
- Today: maximum performance on **tabular** data — trees, ensembles of trees, and principled tuning.
- Gradient-boosted trees remain the strongest tabular baseline, often beating deep nets (Grinsztajn et al., 2022).

Credit-scoring thread: push the Week-3 logistic model ($AUC \approx 0.74$) towards 0.80+.

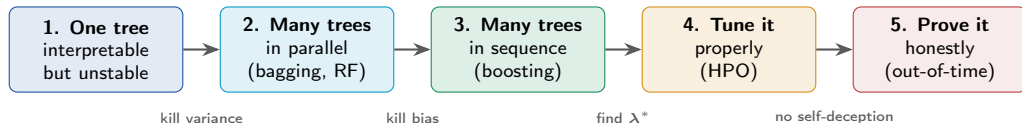


Learning outcomes for today

1. **Describe** a decision tree as an axis-aligned partition of feature space; compute an impurity gain by hand.
2. **Diagnose** why a single tree overfits — the variance problem that everything else today solves.
3. **Contrast** bagging / Random Forests (parallel) with boosting (sequential) and stacking.
4. **Trace** gradient boosting numerically, then read the XGBoost objective.
5. **Apply** LightGBM with monotonic constraints and class weighting to imbalanced credit data.
6. **Design** an HPO study: search space, budget, sampler, pruning — validated honestly.

Lecture 1.5 h → break → Lab 1.5 h (lab04_trees_ensembles_optuna.ipynb)

The story of today in one slide



Everything today is a reaction to a weakness of the previous step.

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

Boosting, Built Up From Intuition

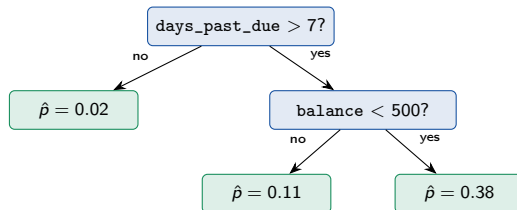
Stacking

Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

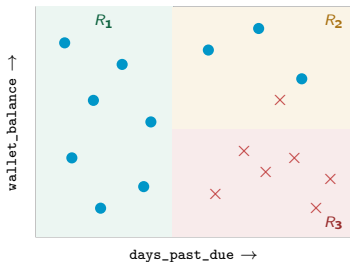
A decision tree is a sequence of questions

- A loan officer's mental checklist, written down and learned from data.
- Each internal node asks one **yes/no question about one feature**.
- Each leaf gives a prediction: a class probability (classification) or a number (regression).
- Prediction cost = depth of the tree. Very fast.



Intuition: a tree is nothing but nested *if* statements that the data chose for us.

Geometrically: trees carve feature space into boxes



- Splits are **axis-aligned**: each cut is perpendicular to one feature axis.
- The leaves are disjoint boxes $R_1, \dots, R_T$ covering the space; the prediction is constant inside each box.
- Consequence: a diagonal boundary needs a *staircase* of many splits — trees are poor at smooth linear structure, excellent at interactions and thresholds.

Intuition: the tree drawing and the box picture are the *same object* viewed two ways.

How a tree is grown: greedy recursive splitting

CART (Breiman et al., 1984). At each node, over all features j and thresholds s :

$$(j^*, s^*) = \arg \min_{j, s} \frac{n_L}{n} I(\text{left}) + \frac{n_R}{n} I(\text{right})$$

where $I(\cdot)$ is an **impurity** measure of a node. Then recurse on each child until a stopping rule fires.



Greedy: the best split *now* is taken, never revisited. Finding the globally optimal tree is NP-hard — greediness is a computational surrender, and part of why single trees are unstable.

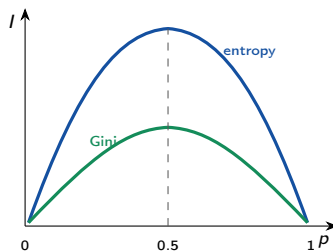
Impurity: how mixed is this node?

For a node with class-1 proportion p :

$$I_{\text{Gini}} = 1 - p^2 - (1 - p)^2 = 2p(1 - p)$$

$$I_{\text{entropy}} = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

- Both are 0 at $p \in \{0, 1\}$ (pure) and maximal at $p = 0.5$ (useless).
- Gini is cheaper (no logs) and is scikit-learn's default; the two rarely disagree in practice.
- Regression uses variance instead:
$$I = \frac{1}{n} \sum_i (y_i - \bar{y})^2.$$



Worked example: scoring one split by hand

Parent node: 1,000 loans, 40 defaults $\Rightarrow p = 0.04$, $I_{\text{Gini}} = 2(0.04)(0.96) = 0.0768$.

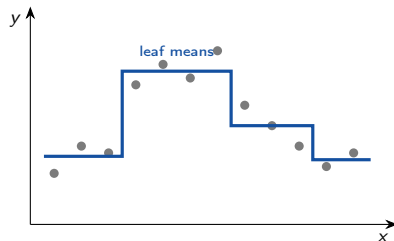
Candidate split	$\text{dpd} > 7$	n	defaults	p	$I_{\text{Gini}} = 2p(1 - p)$
Left ($\text{dpd} \leq 7$)		900	18	0.020	0.0392
Right ($\text{dpd} > 7$)		100	22	0.220	0.3432

$$I_{\text{after}} = \frac{900}{1000}(0.0392) + \frac{100}{1000}(0.3432) = 0.0696 \quad \Rightarrow \quad \Delta I = 0.0072$$

- The tree keeps the split with the largest ΔI among *all* features and thresholds.
- Note the scale:** on a 4%-default problem, even a very informative split yields a tiny absolute gain. Impurity numbers are not business metrics — keep evaluating with PR-AUC and expected profit (Week 3).

Regression trees: same machinery, different leaf

- Impurity $\rightarrow$ within-node variance;
leaf value $\rightarrow$ mean of y in the leaf.
- Prediction surface is a **staircase**:
piecewise constant, never
extrapolates beyond the training
range.
- This staircase is exactly what
gradient boosting will stack up
later in the lecture.



Caution: a regression tree can never predict a value outside the observed range of y — a real limitation for e.g. exposure forecasting.

Trees in scikit-learn, and the knobs that matter

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text

tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,           # complexity cap
    min_samples_leaf=200,  # support per leaf
    ccp_alpha=1e-4,        # cost-complexity
                          pruning
    class_weight="balanced",
    random_state=42,
)
tree.fit(X_train, y_train)

# a tree you can read out loud to a regulator
names = list(X_train.columns)
print(export_text(tree, feature_names=names))

```

Knob	Controls
max_depth	questions deep
min_samples_leaf	leaf-estimate noise
min_samples_split	when to stop
max_features	candidates per split
ccp_alpha	pruning strength

These are our first genuine **hyperparameters** — chosen by us, not learned from data. Section 6 is about choosing them properly.

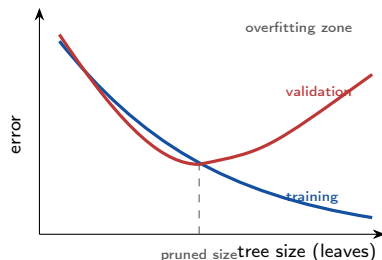
Cost-complexity pruning: grow big, then cut back

Minimal cost-complexity criterion for a subtree T :

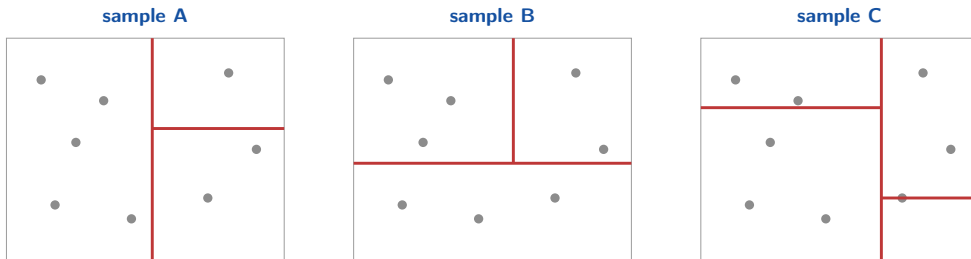
$$R_\alpha(T) = R(T) + \alpha |\tilde{T}|$$

with $R(T)$ the training error and $|\tilde{T}|$ the number of leaves.

- $\alpha = 0$: keep the full grown tree.
- $\alpha \uparrow$: leaves become expensive; weak branches collapse.
- Sweeping α produces a *nested* sequence of trees — pick one by cross-validation.



The fatal flaw: a single tree is a high-variance estimator



- Resample the training data slightly and the tree can change **completely**: greedy splitting means one flipped split at the root rewrites everything below it.
- Low bias (a deep tree can fit anything), **high variance** (it fits the noise too).

This single fact motivates the entire rest of the lecture. Averaging many unstable trees $\Rightarrow$ bagging / RF. Adding many *small* trees $\Rightarrow$ boosting.

Trees: honest scorecard

Strengths	Weaknesses
No scaling or encoding of monotone transforms needed	High variance; unstable under resampling
Captures interactions and thresholds automatically	Axis-aligned: bad at smooth / linear structure
Handles mixed types; missing values natively (LightGBM)	Greedy: no global optimality guarantee
Readable — an adverse-action reason falls out of the path	Biased towards high-cardinality features
Fast inference, tiny memory footprint	A single tree rarely competitive on accuracy

Credit-scoring thread: a depth-4 tree on the Week-2 features reaches ROC-AUC ≈ 0.71 — *below* the logistic baseline of 0.74. Alone, it is not enough.

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

Boosting, Built Up From Intuition

Stacking

Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

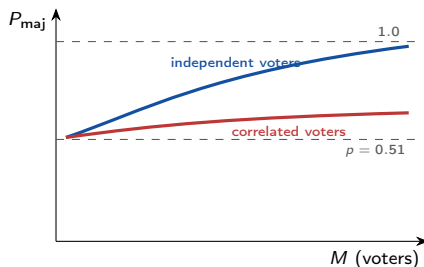
The wisdom of (diverse) crowds

- M **independent** classifiers, each accuracy $p > 0.5$; majority vote:

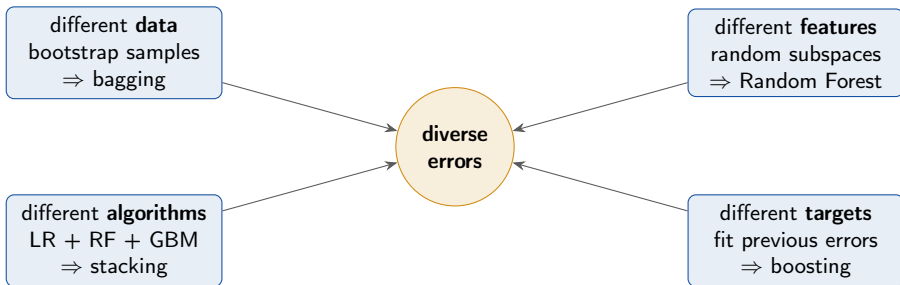
$$P_{\text{maj}} = \sum_{k=\lceil M/2 \rceil}^M \binom{M}{k} p^k (1-p)^{M-k}$$

and $P_{\text{maj}} \rightarrow 1$ as $M \rightarrow \infty$
(Condorcet).

- 1,000 voters with $p = 0.51 \Rightarrow$ majority $\approx 73\%$ — *if errors are independent.*
- Real models are **correlated**; ensembling is the art of manufacturing *diversity*.



Diversity levers: four ways to make trees disagree usefully



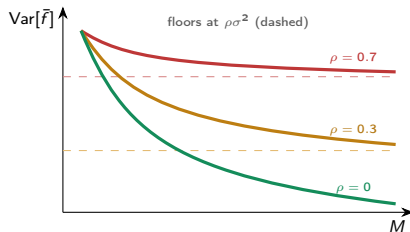
The rest of the lecture is a tour of these four levers, in this order.

The formal reason: variance of an average

For an ensemble average $\bar{f}(x) = \frac{1}{M} \sum_m f_m(x)$ with member variance σ^2 and average pairwise correlation ρ :

$$\text{Var}[\bar{f}(x)] = \rho \sigma^2 + \frac{1 - \rho}{M} \sigma^2$$

- Adding members kills the second term; the **floor** is $\rho \sigma^2$.
- Two levers: $M \uparrow$ (cheap) and $\rho \downarrow$ (the clever part).
- Averaging does **not** reduce bias — that is boosting's job.



The ensemble family tree

Family	Trains	Attacks	Base learner
Bagging / Random Forest	in parallel	variance	deep trees (low bias)
Boosting	sequentially	bias, then variance	stumps / shallow trees
Voting / Stacking	independently	both, via heterogeneity	diverse families

deep trees ||

variance ↓

stumps →→→

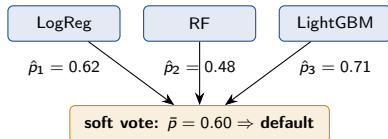
bias ↓

LR + RF + GBM

heterogeneity

Match the base learner to the mechanism: a committee of underfit models is still underfit.

Voting classifiers: the simplest ensemble



- **Soft** voting averages probabilities — confidence is information; needs Week-3 calibration.
- **Hard** voting counts labels only.

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from lightgbm import LGBMClassifier

vote = VotingClassifier(
    estimators=[
        ("lr", LogisticRegression(max_iter=2000)),
        ("rf", RandomForestClassifier(
            n_estimators=400)),
        ("lgbm", LGBMClassifier(n_estimators=600)),
    ],
    voting="soft",          # average probabilities
    weights=[1, 1, 2],      # trust the GBM more
)
vote.fit(X_train, y_train) # X from Week-2
pipeline
```

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

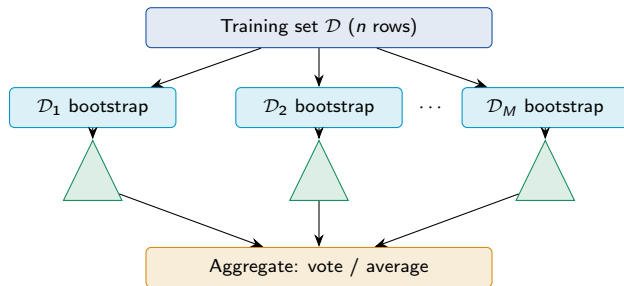
Boosting, Built Up From Intuition

Stacking

Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

Bagging: bootstrap aggregating (Breiman, 1996)



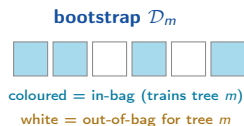
- Sample *with* replacement, size n ; train one **deep** tree per sample; aggregate.
- Trees train **in parallel** (`n_jobs=-1`); boosting will not have this luxury.

Out-of-bag (OOB): free honest validation

- Each bootstrap omits $(1 - \frac{1}{n})^n \approx e^{-1} \approx 37\%$ of the rows.
- Predict row i using only the $\approx 0.37M$ trees that never saw it:

$$\widehat{\text{err}}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n L(y_i, \bar{f}_{-i}^{\text{OOB}}(x_i))$$

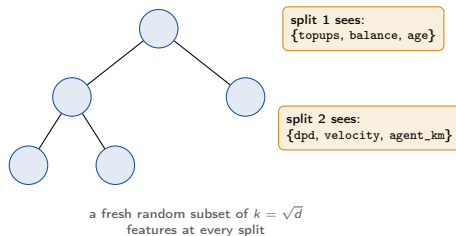
- $\approx$ leave-many-out CV at zero extra cost.



But: OOB uses random bootstraps — it cannot replace the Week-3 *temporal* validation on drifting credit data.

Random Forests: bagging + feature decorrelation

- At *each split*, only a random subset of k features is considered (default $k = \sqrt{d}$).
- Trees stop seeing the same dominant features $\Rightarrow$ correlation ρ drops $\Rightarrow$ the variance floor $\rho\sigma^2$ drops.
- **Extra-Trees:** also randomise the split threshold — faster, sometimes smoother.



Key knobs: `n_estimators` (more $\rightarrow$ plateau, never worse), `max_features`, `min_samples_leaf`.

Random Forest on the credit data

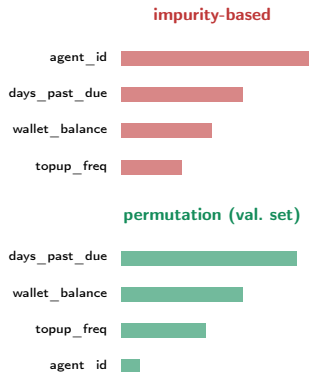
```
from sklearn.ensemble import
    RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=800,
    max_features="sqrt",
    min_samples_leaf=20,      # smooth leaves
    class_weight="balanced", # ~4% defaults
    oob_score=True,          # free validation
    n_jobs=-1, random_state=42,
)
rf.fit(X_train, y_train)
print(f"OOB accuracy: {rf.oob_score_:.3f}")
```

- `min_samples_leaf=20`: leaf probability estimates on 4% default data need support.
- `class_weight="balanced"` reweights the loss by inverse class frequency.
- `oob_score=True` gives the internal estimate — still check the temporal split.

Credit-scoring thread: RF lifts the single tree 0.71 $\rightarrow$ $\approx$ 0.78 ROC-AUC. Same base learner, no new features — pure variance reduction.

Pitfall: impurity importance loves high-cardinality features



- Impurity importance is computed on *training* splits: high-cardinality IDs offer many split points $\Rightarrow$ inflated scores.
- **Permutation importance** on a held-out set measures real predictive contribution.

```
from sklearn.inspection import
    permutation_importance
imp = permutation_importance(
    rf, X_val, y_val,
    scoring="roc_auc", n_repeats=10)
```

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

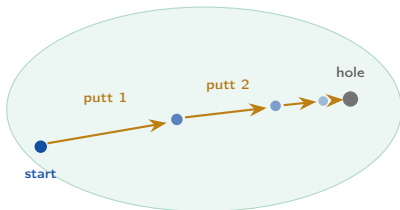
Boosting, Built Up From Intuition

Stacking

Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

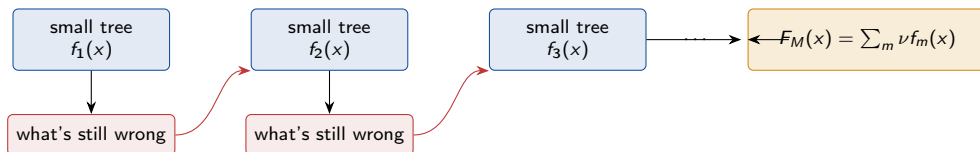
Boosting in one analogy: putting on a golf green



- Each putt aims at the **remaining distance**, not at the hole from scratch.
- Every stroke is small and deliberately *under-hit* — that is the learning rate ν .
- Many small corrections beat one heroic swing, which overshoots (= overfits).
- Stop when you are close enough: **early stopping**.

Intuition: *boosting = fit the model to what is still wrong, then take a small step in that direction, repeat.*

The mechanism, before any mathematics



- Bagging asks M trees the *same* question and averages. Boosting asks each tree a *new* question: “what did the committee so far get wrong?”
- Consequence: strictly **sequential** (no free parallelism), and it attacks **bias**, so the base learners should be weak.

Numeric trace: three boosting rounds on six loans

Regression with squared error; $\nu = 0.5$; each f_m is a single stump splitting on x .

i	y_i	F_0 (mean)	$r_i^{(1)} = y_i - F_0$	f_1 leaf	$F_1 = F_0 + \nu f_1$	$r_i^{(2)}$
1	12	13.3	-1.3	-1.30	12.65	-0.65
2	15	13.3	+1.7	-1.30	12.65	+2.35
3	9	13.3	-4.3	-1.30	12.65	-3.65
4	20	13.3	+6.7	+1.37	13.98	+6.02
5	14	13.3	+0.7	+1.37	13.98	+0.02
6	10	13.3	-3.3	+1.37	13.98	-3.98
Sum of squared errors			79.3			71.3

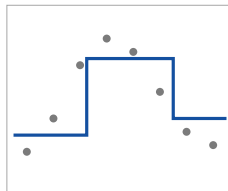
- F_0 is just the mean. The stump splits rows $\{1, 2, 3\}$ from $\{4, 5, 6\}$ and predicts the *mean residual* in each group.
- Because $\nu = 0.5$, we move only halfway — SSE falls $79.3 \rightarrow 71.3$, and round 2 repeats on the new residual column.

The same trace, drawn

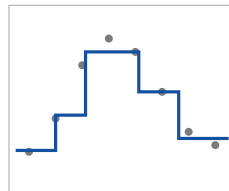
F_0 : the mean



$F_1 = F_0 + \nu f_1$



F_{20} : close fit



- Expressiveness comes from **many shallow** trees, not one deep one — the staircase refines itself.
- Left unchecked it will keep refining until it fits the noise. Hence shrinkage + early stopping, next.

From “residuals” to “gradients”: the one abstraction

We fitted $r_i = y_i - F(x_i)$ because the loss was squared error. Notice:

$$L = \frac{1}{2}(y_i - F(x_i))^2 \implies -\frac{\partial L}{\partial F(x_i)} = y_i - F(x_i)$$

- The residual is the negative gradient of the loss with respect to the prediction.
- So replace “fit the residual” with “fit the **negative gradient**” and the method works for *any* differentiable loss — log-loss, Huber, quantile, ranking.

Loss	Pseudo-residual r_i
squared error	$y_i - F(x_i)$
log-loss (binary)	$y_i - \sigma(F(x_i))$
absolute error	$\text{sign}(y_i - F(x_i))$

Intuition: gradient boosting is gradient descent where the “parameter” being updated is the *function* F itself, and each tree is one step.

Gradient boosting, stated formally

Build $F_M(x) = F_0(x) + \sum_m \nu f_m(x)$ minimising $\sum_i L(y_i, F(x_i))$:

1. $F_0(x) = \arg \min_{\gamma} \sum_i L(y_i, \gamma)$ (a constant: base rate / mean)
2. For $m = 1, \dots, M$, compute pseudo-residuals at the current predictions,

$$r_i^{(m)} = - \left. \frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}$$

3. Fit a small tree f_m to $\{(x_i, r_i^{(m)})\}$; update $F_m = F_{m-1} + \nu f_m$.

Friedman (2001). Everything after this — AdaBoost, XGBoost, LightGBM, CatBoost — is a variation on steps 2 and 3.

AdaBoost: the same idea via row weights

Labels $y_i \in \{-1, +1\}$, weights $w_i^{(1)} = 1/n$.

For $m = 1, \dots, M$:

1. Fit f_m on weighted data; error

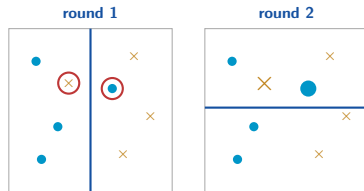
$$\varepsilon_m = \frac{\sum_i w_i^{(m)} \mathbb{I}\{f_m(x_i) \neq y_i\}}{\sum_i w_i^{(m)}}$$

2. Learner weight $\alpha_m = \frac{1}{2} \ln \frac{1-\varepsilon_m}{\varepsilon_m}$

3. Re-weight $w_i^{(m+1)} = w_i^{(m)} e^{-\alpha_m y_i f_m(x_i)}$

Equivalent to stage-wise minimisation of the **exponential loss** $e^{-yF(x)}$ — i.e. a special case of the previous slide.

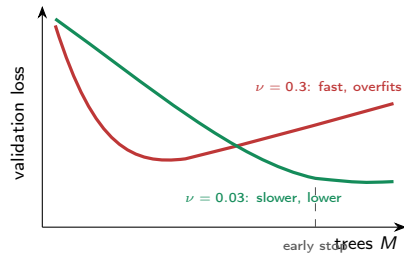
Mistakes are drawn bigger because they now weigh more.



Fragility: exponential loss punishes label noise brutally — disputed defaults and reversed transactions are common in operational credit data. Prefer log-loss gradient boosting.

Shrinkage: why we deliberately under-hit

- Smaller $\nu \Rightarrow$ each tree contributes less $\Rightarrow$ need more trees, but the path is smoother and generalises better.
- (ν, M) is the single most important pair to tune, and they trade off almost exactly.
- Practice: fix M large, tune ν , and let **early stopping** choose the effective M on a validation window.



XGBoost (Chen & Guestrin, 2016): the regularised objective

At stage m , take a second-order Taylor expansion of the loss and add an explicit complexity penalty:

$$\mathcal{L}^{(m)} \approx \sum_{i=1}^n \left[g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2 \right] + \underbrace{\gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2}_{\Omega(f_m): \text{ tree complexity}}$$

with $g_i = \partial_F L(y_i, F)$, $h_i = \partial_F^2 L(y_i, F)$, T leaves, leaf values w_j .

- Regularisation lives *inside the objective*: λ (L2 on leaf values), γ (cost per leaf) — plus row and column subsampling per tree.
- Second-order information (h_i) makes each step a Newton step, not just a gradient step.

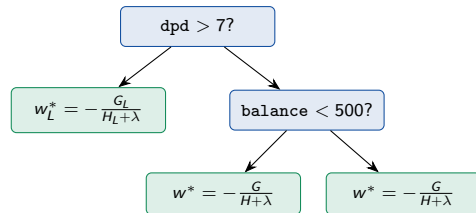
XGBoost: closed-form leaves and the split gain

Optimal value of leaf j (set derivative to zero):

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

Gain of splitting a node into L and R :

$$\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$



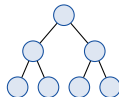
- Split accepted only if Gain > 0 : γ is built-in pre-pruning — compare with the Gini gain we computed by hand in Section 1.

Every leaf outputs a *score* w^* built from gradient statistics — not a class count.

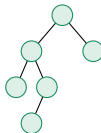
XGBoost vs LightGBM vs CatBoost

	XGBoost	LightGBM	CatBoost
Tree growth	level-wise	leaf-wise (best-first)	symmetric (oblivious)
Speed (large tabular)	fast	fastest	moderate
Categoricals	encode upstream	native	native, ordered stats
Overfitting profile	moderate	aggressive	conservative
Killer feature	maturity, ecosystem	speed on wide data	categorical handling

level-wise



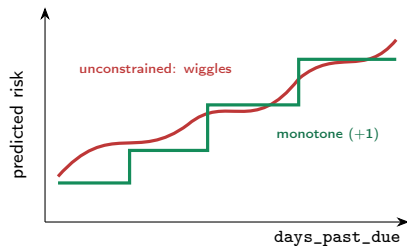
leaf-wise



leaf-wise reaches lower loss for the same number of leaves — and overfits faster: $\text{cap_num_leaves} \ll 2^{\text{max_depth}}$

Regulated credit scoring needs monotonic constraints

- Risk must not *decrease* as delinquency grows — reviewers and adverse-action logic expect it (CBK DCP rules; Kenya DPA 2019 governance).
- Unconstrained trees happily learn local wiggles from noise; constrained trees reject those splits.
- Supported natively by XGBoost, LightGBM and CatBoost.



LightGBM for the credit model, production-grade defaults

```
import lightgbm as lgb

mono = [ +1,  # days_past_due      -> risk up
         -1,  # wallet_balance_kes -> risk down
         0,   # topup_frequency
         0 ] # agent_distance_km

clf = lgb.LGBMClassifier(
    n_estimators=4000, learning_rate=0.03,
    num_leaves=63, min_child_samples=100,
    subsample=0.8, colsample_bytree=0.8,
    reg_lambda=5.0,
    monotone_constraints=mono,
    scale_pos_weight=24.0,  # ~4% defaults
)
clf.fit(X_tr, y_tr,
        eval_set=[(X_val, y_val)], eval_metric="auc",
        callbacks=[lgb.early_stopping(200)])
```

- $n_{\text{estimators}}$ deliberately huge; early stopping on the *temporal* window picks the real M .
- $\text{scale_pos_weight} \approx n_{\text{neg}}/n_{\text{pos}}$ for the $\approx 4\%$ default rate; evaluate with PR-AUC (Week 3).
- Governance knobs (mono) are policy, not hyperparameters.

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

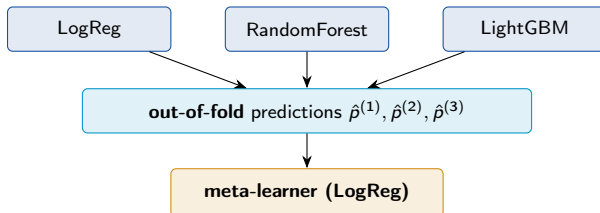
Boosting, Built Up From Intuition

Stacking

Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

Stacked generalisation (Wolpert, 1992)



- Level-1 learns *how to weight* the base models — strictly generalises voting.
- Base predictions must be **out-of-fold**: in-sample predictions would leak the training labels into the meta-learner.

Stacking in scikit-learn — and when it is worth it

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

stack = StackingClassifier(
    estimators=[("lr", lr), ("rf", rf),
                ("lgbm", gbm)],
    final_estimator=LogisticRegression(C=0.5),
    cv=5,
    stack_method="predict_proba",
    n_jobs=-1,
)
stack.fit(X_train, y_train)
```

- Cost: $K \times$ training per base model, plus **every base model in production** at serving time.
- Typical tabular gain: +0.005–0.02 AUC.
- Decision is economic: latency, memory, monitoring burden (Week 13) vs marginal lift.

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

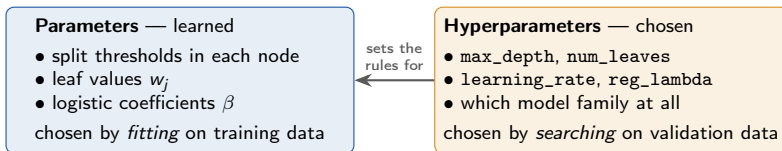
Boosting, Built Up From Intuition

Stacking

Hyperparameter Optimisation, Built Up From Intuition

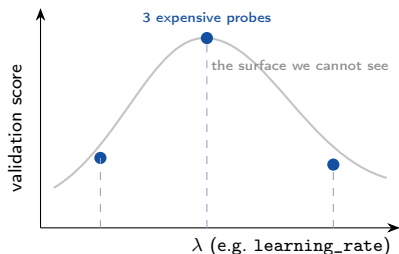
Case Study & Lab

First: what exactly is a hyperparameter?



The gradient tells us how to move parameters. **Nothing** tells us how to move hyperparameters — that is the whole problem.

Why tuning is searching in the dark



Three properties make this unlike model fitting:

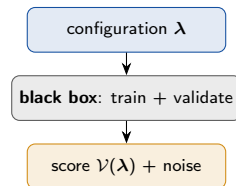
- **Expensive:** one probe = one full train + validate cycle (minutes to hours).
- **No gradients:** the space mixes continuous, integer, categorical and *conditional* dimensions.
- **Noisy:** the same λ evaluated twice gives different scores.

Intuition: you are mapping a landscape by digging a few very expensive holes. Strategy is about *where to dig next*.

The problem, stated

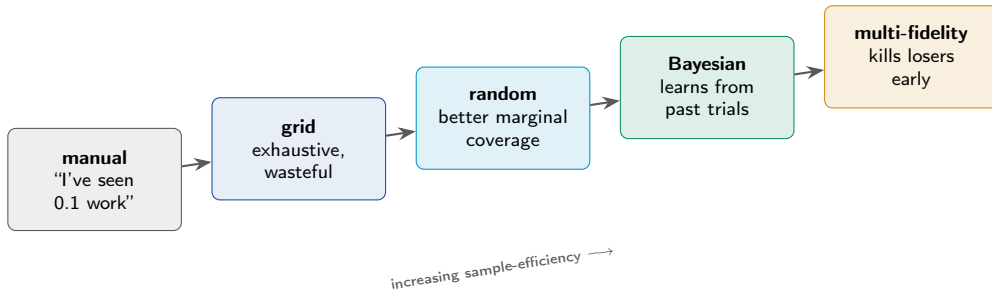
$$\lambda^* = \arg \min_{\lambda \in \Lambda} \mathcal{V}(\mathcal{A}_\lambda(\mathcal{D}_{\text{tr}}), \mathcal{D}_{\text{val}})$$

- $\mathcal{A}_\lambda$: the learning algorithm configured by λ .
- $\mathcal{V}$: the validation metric we actually care about — PR-AUC, expected profit.
- Λ : the search space we design. A bad space cannot be rescued by a good sampler.



The real question: the best configuration findable within a **budget** B of trials / GPU-hours / KES of cloud spend.

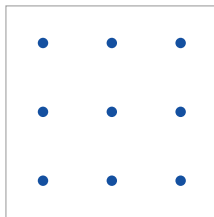
The ladder of search strategies



Each rung answers “where do I dig next?” better than the one before — at the cost of more machinery.

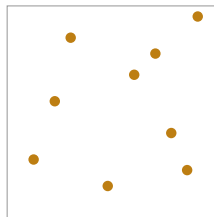
Grid vs random search (Bergstra & Bengio, 2012)

Grid: 9 trials, 3 distinct values



important param: 3 values probed

Random: 9 trials, 9 distinct values



important param: 9 values probed

- Performance is dominated by **few** hyperparameters; grid spends its budget re-probing the unimportant ones.
- Random probes B values *per marginal*, stops anytime, parallelises trivially — the floor for any serious study.

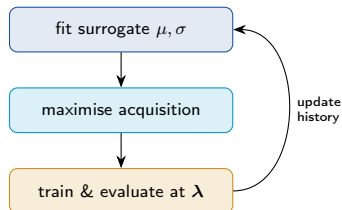
Bayesian optimisation: use what previous trials told you

Random search has no memory — trial 50 is as ignorant as trial 1. Bayesian optimisation fixes that:

1. Fit a cheap **surrogate** of the expensive surface: predicted mean $\mu(\lambda)$ and uncertainty $\sigma(\lambda)$.
2. Choose the next point by maximising an **acquisition function**, e.g. Expected Improvement:

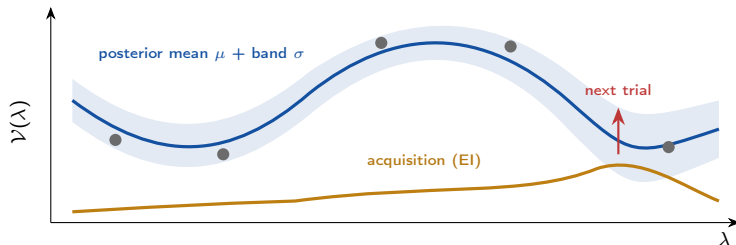
$$\text{EI}(\lambda) = \mathbb{E}[\max(0, \mathcal{V}^* - \mathcal{V}(\lambda))]$$

3. Evaluate, add to history, refit. Repeat.



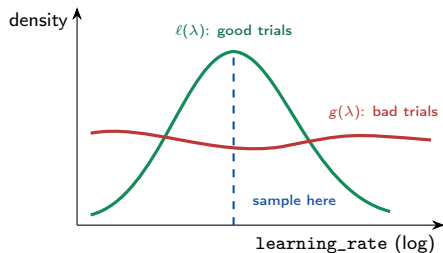
Intuition: explore where σ is large, exploit where μ looks good — the acquisition function arbitrates.

Bayesian optimisation in one picture



Note where the arrow points: not at the current best, but at a promising region we know least about.

TPE: the sampler Optuna actually uses

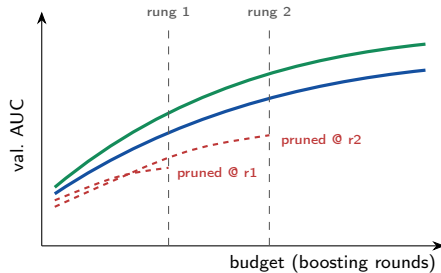


- Split the trial history at a quantile: the top γ fraction is “good”, the rest “bad”.
- Fit two densities over Λ : $\ell(\lambda)$ from the good trials, $g(\lambda)$ from the bad ones.
- Sample candidates and pick the one maximising $\ell(\lambda)/g(\lambda)$ — exactly EI up to a monotone transform.

Intuition: “propose values that look like the ones that worked, and unlike the ones that didn’t.” Handles mixed and conditional spaces — hence Optuna’s default (Bergstra et al., 2011).

Multi-fidelity HPO: stop wasting time on losers

- Bad configurations usually *look bad early* (few boosting rounds, small data subsample).
- **Successive halving**: start N configs on a small budget; keep the best $1/\eta$; multiply their budget by η ; repeat.
- **Hyperband** (Li et al., 2017) hedges over several halving schedules.
- In Optuna a **pruner** does this — routinely a 3–10 $\times$ effective speed-up.



Optuna: define-by-run HPO in ~25 lines

```
import optuna, lightgbm as lgb
from sklearn.model_selection import StratifiedKFold, cross_val_score
def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "num_leaves": trial.suggest_int("num_leaves", 15, 255, log=True),
        "min_child_samples": trial.suggest_int("min_child_samples", 20, 500, log=True),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-2, 50.0, log=True),
        "n_estimators": 2000, "scale_pos_weight": 24.0,
        "monotone_constraints": MONO,      # governance is not tunable!
    }
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    score = cross_val_score(lgb.LGBMClassifier(**params), X_train, y_train,
                           scoring="average_precision", cv=cv, n_jobs=-1)
    return score.mean()                # PR-AUC: imbalanced target
study = optuna.create_study(direction="maximize",
                            sampler=optuna.samplers.TPESampler(seed=42),
                            pruner=optuna.pruners.MedianPruner(n_warmup_steps=5))
study.optimize(objective, n_trials=100, timeout=3600)
print(study.best_params, study.best_value)
```

Designing the search space: the knobs that matter

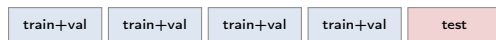
Knob	Typical range
learning_rate	10^{-3} –0.3 (log)
num_leaves	15–255 (log-int)
min_child_samples	20–500 (log-int)
subsample	0.5–1.0
colsample_bytree	0.5–1.0
reg_lambda	10^{-2} –50 (log)

TPE budget heuristic: $n_{\text{trials}} \gtrsim 10 \times$
dimensions; here $6 \Rightarrow 60$ –100 trials.

- **Log-scale** anything spanning orders of magnitude.
- Fix `n_estimators` high; early stopping picks the effective value — do not tune redundant knobs.
- **Freeze governance**: monotonic constraints and cost weights come from policy, not from the sampler.
- After ~ 30 trials, inspect `optuna.importance` (fANOVA) and narrow the space around what matters.
- Pipeline choices (imputer, encoder) are categorical hyperparameters too.

Honest evaluation of a tuned model: nested CV

outer loop



×5: generalisation estimate

inner loop



×4: Optuna picks λ^*

- The inner loop *selects*; the outer loop *scores the whole procedure, search included*.
- Reporting the inner-loop best CV score is optimistically biased — the **winner's curse** of HPO: with 100 trials, the maximum of 100 noisy scores is above the truth by construction.

HPO hygiene: three rules we will not break

1. **Never touch the test set while tuning.** Tune on validation folds; report *once* on the untouched out-of-time window. Tuning on test = overfitting the test set.
2. **Tune the metric you actually care about.** PR-AUC / expected profit for the 4%-default credit problem — not accuracy, not even ROC-AUC.
3. **Log every study.** Optuna storage + MLflow (Week 12). HPO without experiment tracking is unreproducible archaeology.

Industry compromise when nested CV is too expensive: one temporal hold-out for tuning + one final untouched out-of-time test window — exactly what the lab uses (loans from the last 3 months).

Roadmap

Decision Trees: the Building Block

Why Ensembles Work

Bagging and Random Forests

Boosting, Built Up From Intuition

Stacking

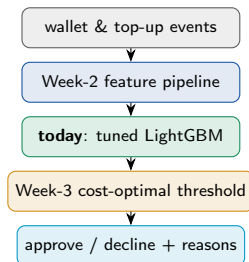
Hyperparameter Optimisation, Built Up From Intuition

Case Study & Lab

Case study: thin-file mobile-money credit scoring, round 3

Setting (Weeks 1–3 continued): a Nairobi fintech underwrites 30-day nano-loans from mobile-money behaviour — no credit-bureau file.

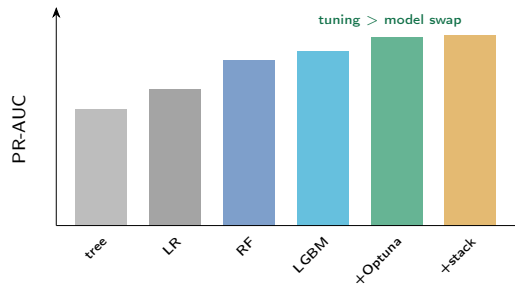
- $n \approx 1.2\text{M}$ loans, $d = 87$ features (Week 2 pipeline), default rate 4.1%, strict temporal split.
- **Constraints:** adverse-action reasons; monotone risk in delinquency; no protected attributes nor proxies.
- **Business metric:** expected profit per 1,000 KES lent at the Week-3 cost-optimal threshold.



Round-3 results: what bought us what

Model	PR-AUC	ROC-AUC
Single tree (d=4)	0.181	0.712
Logistic (W3)	0.212	0.741
Random Forest	0.257	0.779
LightGBM default	0.271	0.791
LightGBM + Optuna	0.294	0.812
+ stacking	0.297	0.814

Illustrative results on the course dataset; you reproduce this table in the lab.



Tuning bought more than the model swap (Melis et al., 2018). Stacking's +0.003: worth a second model in production? Defend it in your memo.

Lab (1.5 h): lab04_trees_ensembles_optuna.ipynb — part A

Deliverable: completed notebook + 10-line decision memo, due before Week 5 (Lab Assignments, 20%).

1. One tree first (15 min)

- Fit a depth-4 tree; print `export_text` and read three decision paths as adverse-action reasons.
- Refit on two bootstrap resamples and diff the root split — see the instability for yourself.

2. Bagging & forests (15 min)

- RF with OOB score; compare permutation vs impurity importance on the `agent_id`-style features.

3. Boosting (25 min)

- LightGBM with early stopping on the temporal validation window.
- Add monotonic constraints; quantify the PR-AUC effect of `scale_pos_weight`.

Lab — part B, and how you will be graded

4. HPO (25 min)

- Design a 6-dimensional log-aware search space; run a 60-trial Optuna TPE study with median pruning.
- Plot `plot_optimization_history` and `plot_param_importances`.

5. Honesty check (10 min)

- Re-score on the untouched out-of-time window; report the optimism gap vs inner-CV PR-AUC.

6. Stretch: stack LR + RF + tuned LightGBM; argue whether the marginal gain justifies serving two extra models.

Rubric component	Weight
Correctness & leakage safety	40%
HPO design quality (space, budget, pruning)	30%
Decision-memo reasoning	30%

Key takeaways

- A decision tree is an axis-aligned partition grown greedily by impurity reduction: readable, fast, and **high variance**. Everything else today is a fix for that last word.
- Ensembles manage $\rho\sigma^2 + \frac{1-\rho}{M}\sigma^2$: bagging/RF cut **variance** via decorrelation; boosting cuts **bias** by fitting what is still wrong, one small step at a time.
- Gradient-boosted trees are the tabular workhorse — and with monotonic constraints, early stopping and explicit regularisation they are also *governable* for regulated credit.
- HPO is budgeted search in the dark: random beats grid; TPE/Bayesian beats random once trials are expensive; pruning multiplies your budget — and the metric you tune is the metric you get.

Reading & next week

Reading

- Course book ch. 4 (this session); Géron ch. 6–7 for extra worked examples.
- Breiman et al. (1984), CART; Breiman (1996), bagging; Friedman (2001), gradient boosting.
- Chen & Guestrin (2016), *XGBoost*; Ke et al. (2017), *LightGBM*.
- Bergstra & Bengio (2012); Akiba et al. (2019), *Optuna*.

Next week (W5, 24 Aug)

- Unsupervised learning & dimensionality reduction: K-Means / DBSCAN wallet segmentation, PCA / t-SNE.

Looking ahead: Optuna returns in Week 6 to tune neural networks, and HPO-in-CI/CD appears in the Week 12 lab.

Reminder: capstone proposal + dataset approval due Week 8 (mid-semester break).